Lego Loam Algorithm Mapping

This algorithm has been successfully compiled and run on the Orin-Nano motherboard. Orin-Nano motherboard system environment: **Ubuntu22.04+ROS2-Humble**.

Startup Command

Open Terminal 1 and enter the following command to start the Lego-Loam mapping program,

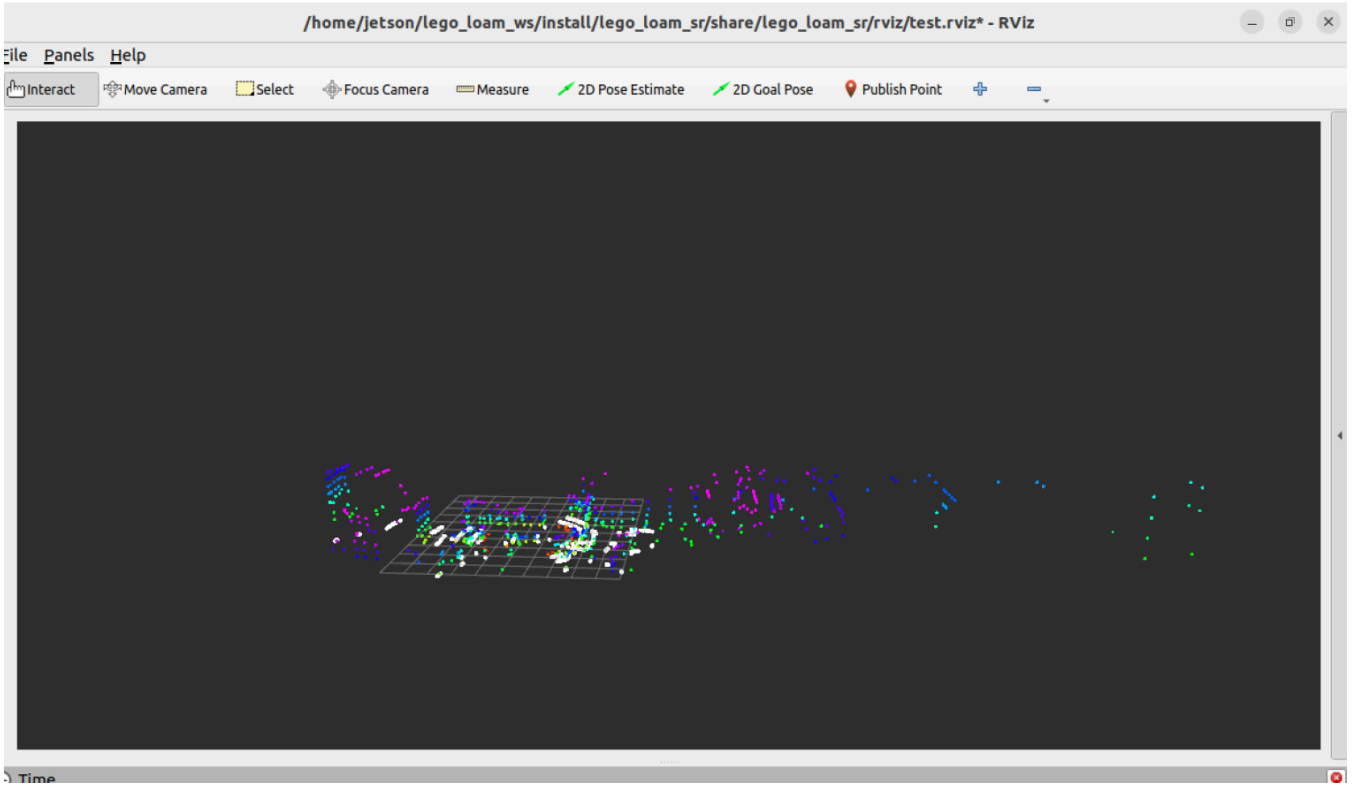
```
ros2 launch lego_loam_sr run.launch.py
```

After the program starts, an rviz will open to display the mapping process.

Open Terminal 2 and enter the following command to start the 32-line lidar,

```
ros2 launch vanjee_lidar_sdk start.py
```

Then, slowly move the lidar. The point cloud map in rviz will be gradually generated, as shown in the following figure,



Saving the Map

Place the save_lego_loam_map.py from the [Attachments] folder in the ~ directory of your motherboard, then open Terminal 3 and enter the following command to start the command to save the point cloud map,

```
python3 save_lego_loam_map.py
```

Wait for the display showing "received xx point cloud data", then press ctrl c to save the map.

Map location: ~/lego_maps/lego_loam_map_xxxx.pcd, where xxxx is the generated timestamp.

Viewing the Point Cloud Map

Install the pcl_viewer tool, the installation command is as follows,

```
sudo apt install ros-humble-pcl-ros -y
```

Then, in the graphical interface, you can use pcl_viewer to view the point cloud map. In the directory where the point cloud map is saved, open a terminal and enter the following command,

```
pcl_viewer lego_loam_map_xxxx.pcd
```

Replace lego_loam_map_xxxx.pcd here with the actual point cloud map filename you want to view.

Mapping Parameter Table

The Lego_Loam mapping parameter table is located in the LeGO-LOAM-ROS2 directory:

```
LeGO-LOAM-ROS2/LeGO-LOAM/config/loam_config.yaml
```

The parameter table contents are as follows:

```
# VLP-16
image_projection:
  ros__parameters:
    laser:
      num_vertical_scans: 32
      num_horizontal_scans: 1200
      vertical_angle_bottom: -25.0          # degrees
      vertical_angle_top: 15.0              # degrees
      sensor_mount_angle: 0.0               # degrees
      ground_scan_index: 16

    image_projection:
      segment_valid_point_num: 5
      segment_valid_line_num: 3
      segment_theta: 45.0                   # decrese this value may improve accuracy

map_optimization:
  ros__parameters:
    mapping:
      enable_loop_closure: true

      surrounding_keyframe_search_radius: 50.0   # key frame that is within n meters from
current pose will be considered for scan-to-map optimization (when loop closure disabled)
```

```
      surrounding_keyframe_search_num: 50        # submap size (when loop closure enabled)

      history_keyframe_search_radius: 7.0        # key frame that is within n meters from
current pose will be considerd for loop closure
      history_keyframe_search_num: 25            # 2n+1 number of history key frames will be
fused into a submap for loop closure
      history_keyframe_fitness_score: 0.3        # the smaller the better alignment

      global_map_visualization_search_radius: 500.0 # key frames with in n meters will be
visualized
      savePCD: true


feature_association:
  ros__parameters:
    laser:
      num_vertical_scans: 32
      num_horizontal_scans: 1200
      scan_period: 0.1                      # seconds

    mapping:
      mapping_frequency_divider: 5

    featureAssociation:
      edge_threshold: 0.05
      surf_threshold: 0.05
      nearest_feature_search_distance: 6.0
```

Debug and modify according to actual situations.